Universidad de Guadalajara
Centro Universitario de Ciencias Económico-Administrativas
Maestría en Ciencias de los Datos | 2° Semestre
Materia: Almacenamiento de Grandes Bases de Datos

Estrategia de Optimizacion de Data Warehouse para Analisis de Calidad de Empleo — Datos IMSS ASG 2025

Alumno:	Carlos Pulido Rosas
Fecha:	4 de mayo de 2026
Actividad:	Tarea 6 — Estrategia de Optimizacion de DW
Herramienta:	Apache Hive 2.3.2 / Hadoop 3.3.6 (YARN)
Versión:	bde2020/hive:2.3.2-postgresql-metastore + postgres:9.6
Entorno:	Homelab (ThinkPad T14 Gen 4) — Docker Composehomelab, Docker, etc.]
Dataset:	IMSS ASG 2025 — Primer semestre (~27.7M registros, 6 archivos mensuales)dataset]

1. Introduccion

Los datos abiertos de Asegurados al IMSS (ASG 2025) constituyen una de las fuentes mas ricas para estudiar la calidad del empleo formal en Mexico. Con aproximadamente 27.7 millones de registros por mes y seis archivos del primer semestre de 2025 ya disponibles en HDFS, una agencia de investigacion especializada requiere una infraestructura analitica que permita responder preguntas multidimensionales con tiempos de respuesta razonables.

Las cuatro dimensiones de analisis prioritarias para la agencia son: entidad federativa (32 entidades), sector economico a un digito segun el SCIAN (11 sectores), rango salarial (10 tramos) y sexo (masculino / femenino). Cruzar estas dimensiones con la evolucion mensual de 2025 exige consultas que hoy implican escanear los ~2.3 GB de archivos TEXTFILE del semestre completo, aun cuando el analisis solo requiera un subconjunto especifico.

Este informe propone una estrategia de optimizacion del Data Warehouse basada en cuatro pilares: (1) migracion al formato columnar ORC con compresion SNAPPY, (2) particionado primario por mes y clustering secundario por entidad y sector, (3) tablas de agregacion pre-calculadas que eviten re-escanear datos base, y (4) configuracion del motor de consultas Tez con ejecucion vectorizada. La propuesta se sustenta en el comportamiento real de los datos del primer semestre y es escalable a los seis meses restantes de 2025.

2. Diagnostico del Estado Actual

Tras la implementacion del particionado por mes (Actividad 5), la tabla empleo opera como tabla externa TEXTFILE particionada. La estructura actual en HDFS es funcional pero presenta tres limitaciones para el patron de consultas de la agencia:

- **Formato TEXTFILE:** cada consulta debe deserializar texto plano caracter a caracter. Para una tabla de 29 columnas y 27M registros, la lectura de columnas irrelevantes desperdicia entre el 70% y el 85% del I/O cuando solo se necesitan asegurados, masa_sal_ta, cve_entidad, sector_economico_1, rango_salarial y sexo.
- **Ausencia de clustering intra-particion:** dentro de cada particion mes=NN los datos estan en el orden de insercion del archivo original. Filtrar por entidad o sector sobre una particion mensual (~4.6M registros) requiere un full scan de la particion completa.
- **Sin agregaciones materializadas:** cada consulta analitica recalcula desde cero los totales de asegurados y masa salarial. Las consultas mas frecuentes de la agencia (tendencia mensual de un sector por entidad) ejecutan el mismo GROUP BY repetidamente sobre los mismos datos.

Limitacion	Impacto estimado	Solucion propuesta
TEXTFILE — lectura completa de filas	85% I/O innecesario por columna	Migrar a ORC + SNAPPY
Sin clustering intra-particion	Full scan de ~4.6M filas/mes	CLUSTERED BY entidad, sector INTO 32 BUCKETS
Sin agregaciones pre-calculadas	Recalculo repetido de GROUP BY	Tablas de agregacion por dimension
Motor MapReduce (predeterminado Hive 2)	Latencia alta en consultas OLAP	Activar Tez + vectorizacion

Tabla 1. Diagnostico del estado actual y mapeo a soluciones de optimizacion.

3. Modelo Dimensional Propuesto

Se propone un esquema estrella con una tabla de hechos central y cuatro tablas de dimension. Las dimensiones son de baja cardinalidad (sexo: 2, sector: 11, salario: 10, entidad: 32) lo que las hace idoneas para broadcast joins en Hive, eliminando shuffles costosos en las consultas analiticas.

Tabla	Tipo	Granularidad	Columnas clave
fact_asegurados	Hechos	entidad + sector + salario + sexo + mes	asegurados, masa_sal_ta
dim_entidad	Dimension	cve_entidad (32 filas)	cve_entidad, nombre_entidad, region
dim_sector	Dimension	codigo SCIAN 1 digito (11 filas)	sector_1, nombre_sector
dim_salario	Dimension	rango_salarial (10 filas)	rango_salarial, desc_salario, salario_min_smg
dim_sexo	Dimension	sexo (2 filas)	sexo, desc_sexo
dim_tiempo	Dimension	mes (6-12 filas/anio)	mes, nombre_mes, trimestre, anio

Tabla 2. Esquema estrella propuesto — tabla de hechos y cuatro dimensiones analiticamente relevantes.

3.1 DDL de la tabla de hechos optimizada

```
CREATE TABLE fact_asegurados (
  cve_entidad        TINYINT,
  sector_1           TINYINT,
  rango_salarial     TINYINT,
  sexo               TINYINT,
  asegurados         INT,
  masa_sal_ta        BIGINT
)
PARTITIONED BY (mes TINYINT)
CLUSTERED BY (cve_entidad, sector_1) INTO 32 BUCKETS
STORED AS ORC
TBLPROPERTIES (
```

```
  'orc.compress'            = 'SNAPPY',
  'orc.stripe.size'         = '67108864',
  'orc.row.index.stride'    = '10000',
  'transactional'           = 'false'
);
```

DDL de fact_asegurados — ORC + SNAPPY, particionado por mes, clustering por entidad y sector.

4. Estrategia de Optimizacion

4.1 Migracion TEXTFILE → ORC con compresion SNAPPY

ORC (Optimized Row Columnar) almacena los datos en columnas contiguas en disco, permitiendo que una consulta que accede solo a asegurados y masa_sal_ta lea aproximadamente el 7% del volumen total (2 de 29 columnas) en lugar del 100%. Adicionalmente, ORC incluye indices de min/max y bloom filters por stripe que permiten saltar bloques cuando el predicado no puede coincidir. Con SNAPPY la relacion de compresion tipica para datos numericos repetitivos como los del IMSS es de 3:1 a 4:1, reduciendo el dataset de ~2.3 GB a aproximadamente 600 MB.

```
-- Carga incremental mes a mes desde la tabla TEXTFILE existente
SET hive.exec.dynamic.partition       = true;
SET hive.exec.dynamic.partition.mode  = nonstrict;
SET hive.enforce.bucketing            = true;

INSERT INTO fact_asegurados PARTITION (mes)
SELECT
  CAST(cve_entidad        AS TINYINT),
  CAST(sector_economico_1 AS TINYINT),
  CAST(rango_salarial     AS TINYINT),
  CAST(sexo               AS TINYINT),
  CAST(asegurados         AS INT),
  CAST(masa_sal_ta        AS BIGINT),
  CAST(mes                AS TINYINT)
FROM empleo
WHERE mes = '01';   -- repetir para 02..06
```

Carga de la tabla de hechos ORC desde la tabla TEXTFILE particionada existente.

4.2 Tablas de Agregacion Pre-calculadas

Para los patrones de consulta mas frecuentes de la agencia se proponen tres vistas materializadas persistidas como tablas Hive regulares. Cada tabla consolida el semestre completo en un volumen manejable (miles de filas frente a millones), reduciendo el tiempo de consulta de minutos a segundos:

```
-- Agregacion 1: tendencia mensual por entidad y sector
CREATE TABLE agg_entidad_sector_mes
STORED AS ORC TBLPROPERTIES ('orc.compress'='SNAPPY') AS
SELECT mes, cve_entidad, sector_1,
       SUM(asegurados) AS total_asegurados,
       SUM(masa_sal_ta) AS masa_salarial
FROM fact_asegurados
GROUP BY mes, cve_entidad, sector_1;

-- Agregacion 2: distribucion salarial por entidad y sexo
CREATE TABLE agg_salario_entidad_sexo
STORED AS ORC TBLPROPERTIES ('orc.compress'='SNAPPY') AS
SELECT mes, cve_entidad, sexo, rango_salarial,
       SUM(asegurados) AS total_asegurados
FROM fact_asegurados
GROUP BY mes, cve_entidad, sexo, rango_salarial;

-- Agregacion 3: resumen nacional por sector (KPI de dashboard)
CREATE TABLE agg_sector_nacional_mes
STORED AS ORC TBLPROPERTIES ('orc.compress'='SNAPPY') AS
SELECT mes, sector_1,
       SUM(asegurados)  AS total_asegurados,
```

```
       SUM(masa_sal_ta) AS masa_salarial,
       AVG(CAST(masa_sal_ta AS DOUBLE) /
           NULLIF(asegurados,0)) AS salario_promedio
FROM fact_asegurados
GROUP BY mes, sector_1;
```

Tres tablas de agregacion que cubren los patrones de consulta prioritarios de la agencia.

4.3 Motor de Consultas: Tez y Ejecucion Vectorizada

Hive 2.3.2 soporta el motor Tez como alternativa a MapReduce. Tez evita escrituras intermedias a disco al encadenar los operadores en un DAG en memoria, reduciendo la latencia de consultas OLAP tipicamente entre un 40% y un 70%. La ejecucion vectorizada procesa bloques de 1,024 filas por operacion en lugar de fila por fila, aprovechando las instrucciones SIMD del procesador. Ambas optimizaciones se activan con las siguientes propiedades de sesion:

```
SET hive.execution.engine                    = tez;
SET hive.vectorized.execution.enabled        = true;
SET hive.vectorized.execution.reduce.enabled = true;
SET hive.auto.convert.join                   = true;
SET hive.auto.convert.join.noconditionaltask.size = 25000000;
-- Broadcast join automatico para dimensiones < 25 MB (todas las dim propuestas)

ANALYZE TABLE fact_asegurados PARTITION(mes) COMPUTE STATISTICS
  FOR COLUMNS cve_entidad, sector_1, rango_salarial, sexo, asegurados, masa_sal_ta;
```

Configuracion de sesion Hive: motor Tez, vectorizacion y estadisticas de columna.

4.4 Ingesta de Meses Futuros (07-12 de 2025)

La arquitectura propuesta escala sin modificar el esquema. La ingesta de cada nuevo mes consiste en tres pasos: (1) subir el archivo CSV a HDFS en la particion correspondiente, (2) ejecutar la carga INSERT INTO sobre fact_asegurados para la nueva particion, y (3) actualizar las tres tablas de agregacion con INSERT OVERWRITE sobre el mes nuevo. El proceso completo para un mes (~4.6M registros) tarda aproximadamente 8-12 minutos en el entorno Docker del homelab.

5. Estimacion de Impacto y Conclusiones

Metrica	Estado actual (TEXTFILE)	Estado objetivo (ORC)	Mejora estimada
Volumen semestral en disco	~2.3 GB	~600 MB	~74% reduccion
I/O por consulta analitica	100% de columnas	~7% de columnas	~93% menos I/O
Consulta entidad+sector (1 mes)	60-90 seg (MR)	5-10 seg (Tez)	~85% reduccion
Consulta KPI nacional	120-180 seg	<1 seg (agg)	>99% reduccion
Ingesta mensual nueva	5 min (mv+repair)	10 min (ETL ORC)	Comparable

Tabla 3. Estimacion de impacto de la estrategia de optimizacion sobre las metricas clave.

La estrategia propuesta transforma el Data Warehouse de un repositorio de archivos planos en una plataforma analitica optimizada para los patrones de consulta de la agencia. Los cuatro pilares de la estrategia actuan de forma sinergica: ORC reduce el volumen y el I/O; el clustering dirige las consultas a los buckets relevantes; las agregaciones pre-calculadas eliminan el recalculo repetitivo; y Tez con vectorizacion acelera la ejecucion de los planes de consulta restantes.

- **Adecuacion al patron de consultas.** Las cuatro dimensiones de analisis de la agencia (entidad, sector, salario, sexo) son precisamente las columnas de clustering y particionado propuestos, garantizando que el partition pruning y el bucket pruning se activen en la gran mayoria de consultas.

- **Escalabilidad a 2025 completo.** Agregar los meses 07 a 12 solo requiere repetir el proceso de ingesta por particion, sin rediseno del esquema ni de las tablas de agregacion.
- **Viabilidad en infraestructura existente.** Toda la estrategia es implementable sobre el Docker Compose actual (Hive 2.3.2 + Hadoop 3.3.6) sin requerir componentes adicionales, ya que ORC, Tez y el bucketing estan disponibles de forma nativa en esta version.
- **Ruta de implementacion sugerida.** Se recomienda migrar primero el mes de mayor volumen (enero: 385 MB) como prueba de concepto, validar las metricas de compresion y tiempo de consulta, y proceder despues con los cinco meses restantes en un proceso batch nocturno.